Week 2 Report: Classical Models for Anomaly Detection

Author: Abhishek Adbhute (ML Engineering Intern) | Week 2, October 2026 | For: Attenomy ML Lead | Repository: https://github.com/abhishekadbhute-create/abhishek-attenomy-ml-internship

1. Summary

- A tuned **XGBoost** model beats a logistic-regression baseline in repeated cross-validation (PR-AUC 0.266 against 0.220, paired gain +0.045, 95% interval +0.020 to +0.071). The gain is real but modest, and it held on the untouched validation split (0.246 against 0.203).
- **The model's skill comes almost entirely from one column, error_code.** Without it, PR-AUC falls by 54% (0.272 to 0.126). The simple rule 'alert whenever an error code is present' gives precision 0.207 and recall 0.422; the model gives 0.227 and 0.422.
- **58% of anomalies (85 of 147) carry no error code and are almost never caught** (recall 0.047). Their features look like normal calls, so the nine current features contain nothing to detect them with.
- Raw scores are not probabilities (mean 0.375 against a true rate of 0.039). After Platt scaling and a chosen threshold, the model flags 73 of every 1,000 calls and about 23% of alerts are real.

2. What was built

- **Baselines (Day 6):** a majority-class predictor scores 96.1% accuracy but PR-AUC 0.039, so accuracy is misleading and PR-AUC is the main metric. Every model must beat 0.04.
- **Four classical models (Day 7)** were compared under identical stratified 5-fold cross-validation, with and without error_code, and reruns were checked for reproducibility.
- **Feature engineering (Day 8):** stateless features (rates, time of day, account history from earlier calls only) and stateful ones (frequency and target encoding of account_id, quantile bins) refit inside every fold. Adding or dropping each feature group for logistic regression and random forest (52 comparisons) showed no clear effect in 42, small gains in 7 and losses in 3; the largest change was +0.015 PR-AUC and no group helped consistently. Tuned XGBoost and LightGBM scored 0.269 to 0.272 on base, stateless and all features, so the nine raw features were kept.
- **Tuning (Day 9):** XGBoost and LightGBM, 25 random-search trials each, 5-fold stratified CV on PR-AUC, everything inside the leak-free pipeline. XGBoost was chosen for simplicity (it ties LightGBM: difference -0.0007, p = 0.93). The model is saved and reloads in a fresh process with identical predictions.

3. Evidence for the chosen model

Model	PR-AUC (95% interval)	ROC-AUC	Note
Majority class	0.039	0.500	Accuracy 96.1%, catches nothing
Stratified dummy	0.040	0.507	Random guessing at the base rate
Logistic regression	0.220 [0.179, 0.262]	0.807	Baseline
XGBoost (tuned)	0.266 [0.212, 0.319]	0.813	Chosen; +0.045 vs baseline, p = 0.002
LightGBM (tuned)	0.266 [0.216, 0.316]	0.814	+0.046 vs baseline, p = 0.001

Dummy rows are 5-fold CV means; the other rows use fresh 5x3 repeated CV with Nadeau-Bengio corrected intervals, because overlapping training sets make plain intervals too narrow. ROC-AUC differences (+0.006, +0.007) are not significant. With about 680 positive training rows, fold-to-fold spread is about 0.04, so the Gate 2 target of a spread below 0.5% is not reachable, and intervals are reported instead.

4. What the model relies on

- **SHAP** (mean absolute value, summed per original column): error_code 0.558, payload_size_kb 0.211, retry_count 0.196, latency_ms 0.038; every other column is below 0.01. Any error code pushes the anomaly score up and a clean call pushes it down, which matches domain logic.
- **Ablation, same folds:** with error_code PR-AUC 0.2715 and ROC-AUC 0.817; without it 0.1261 and 0.666. The model still beats random (0.126 against 0.039) but is weak.
- **Calibration:** raw scores are inflated, most likely by the class weighting used in tuning (mean 0.375; Brier 0.155, worse than always guessing the base rate at 0.0377). Platt scaling, fitted on out-of-fold training scores only, fixes the scale (mean 0.0389, Brier 0.0332, log loss 0.498 to 0.131) and keeps the ranking. Isotonic regression was not used: similar Brier score but ties lowered PR-AUC.

5. Threshold and alert volume

Validation split (3,750 calls, 147 anomalies)	Alerts per 1,000	Precision	Recall
Model, threshold 0.1834 (max F1, chosen on training out-of-fold scores)	72.8	0.227 [0.179, 0.278]	0.422 [0.344, 0.503]
Rule: alert whenever an error code is present	79.7	0.207	0.422
Random flagging	-	0.039	-

At the chosen threshold the model catches 62 anomalies, misses 85 and raises 211 false alarms. A stricter cut-off trades recall for precision: flagging the top 2% of calls gives validation precision 0.39 and recall 0.16; the top 1% gives 0.49 and 0.14. Which point is right is a business decision about reviewer capacity.

6. Where it fails and who it fails for

- **Missed anomalies (85):** 95% have no error code, and their payload (median 16.5 KB), latency (68.7 ms) and retries (0) match correctly ignored normal calls (16.0 KB, 70.6 ms, 0).
- **False alarms (211):** 97% carry an error code. The highest-scoring ones all do, and 7 of the top 8 have 3 retries, so they look like real anomalies. Rate-limit errors are over-represented among false alarms (precision 0.176 against 0.26 to 0.28 for other codes), a lead worth checking, though that group has only 13 anomalies.
- **Subgroup audit** (channel, region, client tier, interaction type, retry band, error code; 95% Wilson intervals): 4 of 23 groups differ from the overall result. Two are the error-present and no-error groups. Calls with 2 or more retries have precision 0.395 (their anomaly rate is 9.5% against 3.9%). Region us-east-1 has recall 0.554, which is likely noise given 46 intervals checked. No gap was found elsewhere, but intervals are wide (13 to 62 anomalies per group), which is absence of evidence, not proof of fairness.

7. Risks and recommendations

Risk	Evidence	Recommended action
Dependence on error_code, possible label leakage	PR-AUC falls 54% without it; 95% of false alarms and 94% of caught anomalies carry one.	Confirm with the data owner that error_code is known before the anomaly decision. If it is only known afterwards, the model cannot be used to score calls in advance.
Most anomalies undetected	85 of 147 anomalies have no error code; recall for them is 0.047.	Collect signals the nine features lack, to be defined with the data owner. Account history and time features gave no gain.
Uncalibrated scores	Mean raw score 0.375 against a true rate of 0.039.	Always apply the saved Platt scaling (models/threshold.json) before reading a score as a probability or applying the threshold.
Small evaluation sample	147 validation anomalies; the 95% intervals on precision and recall are 0.10 and 0.16 wide.	Treat single-split numbers as noisy; keep the test split untouched until the final check.

Questions for review: (1) Is error_code known at the moment a call would be scored? (2) What reviewer capacity, in alerts per 1,000 calls, should set the threshold? (3) Can additional signals be provided for anomalies that look like normal calls?